Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives, structures de contrôle et algorithmique fondamentale en C

Synthèse exécutive : Ce cours détaille le fonctionnement, la syntaxe et les critères de choix des trois structures itératives fondamentales (while, do-while, for) en algorithmique et en langage C. L'objectif principal est de permettre l'automatisation du traitement répété d'instructions sans duplication de code, conformément au principe DRY (Don't Repeat Yourself). Un accent particulier est mis sur la gestion rigoureuse des conditions d'entrée et de sortie, l'initialisation et la mise à jour des variables de contrôle, ainsi que la prévention systématique des boucles infinies et des erreurs de syntaxe courantes en contrôle continu.

Utilité des structures itératives et principe DRY

En programmation, la répétition d'instructions ne doit jamais être traitée par copier-coller. Si un algorithme doit traiter 10 000 éléments ou réitérer une demande d'entrée, dupliquer la ligne de code rend le programme lourd, illisible, impossible à maintenir et extrêmement vulnérable aux erreurs. Les structures itératives (ou boucles) permettent de répéter l'exécution d'un bloc d'instructions tant qu'une condition logique (expression booléenne) reste vraie. Pour qu'une boucle fonctionne correctement, elle doit impérativement comporter une condition de sortie valide et une mise à jour effective des variables qui composent cette condition. Sans cette mise à jour, la condition reste bloquée à l'état vrai ('true'), ce qui provoque une boucle infinie. En TP et en examen, l'absence de mise à jour de la variable de contrôle est sévèrement sanctionnée car elle gèle le programme.

- Principe DRY (Don't Repeat Yourself) : interdiction fondamentale de dupliquer du code pour exécuter une tâche répétitive.
- Condition d'arrêt : expression booléenne évaluée pour déterminer la poursuite ou la sortie de la boucle.
- Variable de contrôle / Compteur : variable modifiée au fil des itérations pour faire évoluer la condition d'arrêt.
- Boucle infinie : dysfonctionnement où la condition reste indéfiniment vraie, bloquant le programme et saturant le processeur.

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'attrape et l'emballe (instruction). Si le capteur ne détecte plus de colis ou si le quota est atteint, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide et finirait par surchauffer et détruire la machine.*

La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est la boucle de référence lorsque le nombre exact d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique (ex: une saisie utilisateur, la lecture d'un fichier jusqu'à la fin, ou la réception d'un signal réseau). Puisque l'évaluation se fait avant d'entrer dans le corps de la boucle, si la condition est fausse dès le premier test, le bloc interne est totalement ignoré (0 itération). Il est crucial d'assurer l'initialisation des variables avant l'entrée dans le 'while', et leur modification explicite à l'intérieur du corps de la boucle pour garantir qu'à un moment donné la condition deviendra fausse.

- Pré-conditionnement : évaluation du test logique AVANT chaque exécution du bloc d'instructions.
- Intervalle d'itérations : de 0 à N passages (le bloc peut ne jamais s'exécuter si la condition est fausse dès le début).
- Initialisation préalable : toute variable utilisée dans la condition doit posséder une valeur valide avant l'instruction while.

Cas concret enseignant : *Exemple de la vérification du code PIN à un distributeur de billets : le distributeur demande le code TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'usager tape le bon code du premier coup, la boucle s'arrête immédiatement sans répéter la demande.*

La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence de la boucle 'while', la structure 'do-while' garantit que les instructions contenues dans son corps seront exécutées au moins une fois avant le premier test logique. Elle est principalement dédiée à la saisie sécurisée de données (validation de bornes) et à la gestion de menus interactifs où l'action doit être proposée une première fois avant de demander si l'utilisateur souhaite continuer. En langage C, la syntaxe exige un point-virgule après l'expression du `while(condition);`. L'oubli de ce point-virgule constitue l'une des erreurs de compilation les plus récurrentes chez les étudiants de BUT 1.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution : la boucle s'exécute au minimum 1 fois (intervalle de 1 à N itérations).
- Syntaxe C stricte : obligation absolue de placer un point-virgule ';' après la parenthèse fermante du while : do { ... } while(condition);

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure parfaitement adaptée.'*

La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la syntaxe des boucles en condensant sur une seule ligne les trois étapes indispensables à l'itération : 1) l'initialisation de la variable de contrôle, 2) la condition logique de maintien, et 3) la mise à jour (incrémentation ou décrémentation). Elle est le choix algorithmique privilégié quand le nombre d'itérations est déterminé à l'avance (ex: parcourir un tableau de taille N ou effectuer une opération exactement 50 fois). Bien que les trois composantes situées entre les parenthèses soient optionnelles sur le plan syntaxique en langage C, les omettre nuit gravement à la lisibilité et est considéré comme une très mauvaise pratique d'ingénierie logicielle.

- En-tête condensé : regroupe l'initialisation, la condition de maintien et l'incrémentation sous la forme for(init; cond; inc).
- Usage privilégié : réservé aux situations où le nombre d'itérations est connu avant d'entrer dans la boucle.
- Portée (scope) : possibilité de déclarer la variable de contrôle directement dans l'en-tête (ex: for(int i = 0; ...)).

Cas concret enseignant : *Métaphore du tour de piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur au tour 1 (init), vous vérifiez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (inc).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si la variable de contrôle 'i' n'est pas incrémentée dans le corps de la boucle, la condition de passage (par exemple i < 10) reste constamment vraie. Le programme s'enferme alors dans une boucle infinie, consommant 100% d'un cœur CPU et ne rendant jamais la main. En C, il faut toujours s'assurer qu'au moins une variable de la condition est modifiée à chaque passage.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Sur le plan algorithmique strict, oui. Toute boucle 'for' peut être réécrite sous la forme d'un 'while' en plaçant l'initialisation juste avant et l'incrémentation à la fin du corps. Cependant, la convention et la propreté du code imposent d'utiliser 'for' dès que le nombre d'itérations est connu d'avance, et 'while' lorsque la fin dépend d'un événement imprévisible.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit obligatoirement valider que N est un entier strictement positif (> 0) à l'aide d'une boucle 'do-while', puis calculer la somme au moyen d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables entières n, somme (initialisée à 0) et i. Étape 2 : Mettre en place la saisie sécurisée avec la structure do-while : do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); Étape 3 : Réinitialiser somme = 0 avant le calcul. Étape 4 : Écrire la boucle for pour effectuer le cumul de 1 jusqu'à N : for (i = 1; i <= n; i++) { somme += i; } Étape 5 : Afficher le résultat avec printf("La somme des %d premiers entiers est %d\n", n, somme);

Résultat : Pour un exemple où l'utilisateur saisit d'abord -2 puis 5 : La boucle do-while rejette -2 et redemande la saisie. Lorsque 5 est saisi, la boucle do-while se termine. La boucle for effectue 5 itérations (i=1 -> somme=1 ; i=2 -> somme=3 ; i=3 -> somme=6 ; i=4 -> somme=10 ; i=5 -> somme=15). Le programme affiche correctement 'La somme des 5 premiers entiers est 15'.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' est à pré-condition (test effectué avant l'exécution, donc 0 à N itérations possible), tandis que la boucle 'do-while' est à post-condition (test effectué après l'exécution, garantissant au moins 1 itération).

Q : Que se passe-t-il si la condition d'une boucle 'for' est fausse dès le premier test ?

R : La boucle ne s'exécute aucune fois (0 itération). Le programme saute directement à l'instruction qui suit le bloc de la boucle 'for'.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule final ';' situé juste après la parenthèse fermante de la condition while : do { ... } while (condition);